



PROYECTO DE FIN DE GRADO - DESARROLLO DE APLICACIONES MULTIPLATAFORMA (DAM)

TÍTULO: GymManager: Digitalización y Gestión Integral de Centros Deportivos

AUTOR: Santiago Arenas Mira

CICLO: Grado Superior en Desarrollo de Aplicaciones Multiplataforma (DAM) **TUTOR:** Elam

MODALIDAD: 3 - Desarrollo de Productos

FECHA: Mayo 2026

1. ÍNDICE

1. ÍNDICE	1
2. RESUMEN (ABSTRACT)	1
3. MARCO TEÓRICO / FUNDAMENTACIÓN	2
4. DESARROLLO DEL PROYECTO	3
4.1. Fases del Desarrollo	3
4.2. Recursos y Herramientas	3
4.3. Validaciones y Pruebas	3
5. RESULTADOS, EVALUACIÓN Y CONCLUSIONES	4



5.1. Resultados Obtenidos	4
5.2. Autoevaluación	4
5.3. Conclusiones y Aprendizaje	4

2. RESUMEN (ABSTRACT)

El proyecto **GymManager** nace de la necesidad de modernizar y digitalizar la gestión operativa de gimnasios y centros deportivos locales. A menudo, estos centros dependen de procesos manuales o herramientas fragmentadas que dificultan la trazabilidad de los socios, los pagos y el progreso deportivo.

Este trabajo presenta una solución multiplataforma integral compuesta por tres pilares: un **backend Robusto** desarrollado con Spring Boot que centraliza la lógica de negocio y la persistencia de datos; un **dashboard web** para la administración de clientes, pagos y reportes; y una **aplicación móvil nativa en Android** para que los socios consulten sus rutinas, realicen reservas y sigan su progreso. La finalidad principal es optimizar la administración del centro y mejorar la experiencia del usuario final mediante la sincronización en tiempo real y la automatización de tareas administrativas.



3. MARCO TEÓRICO / FUNDAMENTACIÓN

Para el desarrollo de GymManager se han seleccionado tecnologías líderes en el sector industrial, basándose en criterios de escalabilidad, seguridad y eficiencia:

- Spring Boot (Java 17): Se eligió sobre Java puro por su capacidad de autoconfiguración y el ecosistema de Spring Data JPA y Spring Security. Esto permite un desarrollo ágil de la API REST y una gestión de seguridad robusta mediante JWT.
- MySQL: Como sistema de gestión de bases de datos relacionales, MySQL ofrece la madurez necesaria para manejar relaciones complejas entre clientes, pagos y rutinas, asegurando la integridad referencial y la consistencia de los datos.
- Android (Kotlin): El uso de Kotlin y el desarrollo nativo garantiza un rendimiento óptimo y acceso total a las APIs del dispositivo (como la cámara para QR o notificaciones), superando a las soluciones híbridas en fluidez de interfaz.
- Retrofit: Se emplea para la comunicación entre la aplicación Android y el servidor backend, facilitando la conversión automática de JSON a objetos Kotlin.
- JasperReports: Implementado en el backend para la generación de reportes profesionales en formato PDF (listado de socios, recibos de pago).



4. DESARROLLO DEL PROYECTO

4.1. Fases del Desarrollo

1. Análisis y Diseño de Datos: Definición del esquema relacional en MySQL, identificando las entidades clave (Usuario, Cliente, Pago, Rutina, Ejercicio).
2. Implementación del Backend: Creación de la API REST con Spring Boot. Desarrollo de controladores para cada entidad y configuración de Spring Security para proteger los endpoints.
3. Desarrollo del Frontend Web: Creación de un panel administrativo mediante HTML5, CSS3 y JavaScript, comunicándose con la API para la gestión de CRUDs.
4. Desarrollo de la App Móvil: Implementación de la lógica en Android/Kotlin, integrando Retrofit para el consumo de datos y Room para la caché local.
5. Integración y Pruebas: Sincronización de todos los módulos y pruebas de flujo de datos extremo a extremo.

4.2. Recursos y Herramientas

- IDEs: IntelliJ IDEA (Backend), Android Studio (Mobile), Visual Studio Code (Web).
- Servidor de Base de Datos: XAMPP / MySQL.



- Control de versiones: Git y GitHub.
- Pruebas de API: Postman / Thunder Client.
- Librerías Clave: Room, Retrofit, Glide, JasperReports, Spring Security.

4.3. Validaciones y Pruebas

Se realizaron pruebas de integración para confirmar que las acciones realizadas en una plataforma se reflejan inmediatamente en las otras: * Validación CRUD: Comprobación de que al registrar un nuevo cliente en el Dash board Web, este puede iniciar sesión inmediatamente en la App Android. * Sincronización de Pagos: Verificación de que los pagos registrados por administración aparecen en el historial de la aplicación del usuario. * Generación de QR: Validación del acceso al gimnasio mediante el escaneo de códigos generados dinámicamente en la app.

5. RESULTADOS, EVALUACIÓN Y CONCLUSIONES

5.1. Resultados Obtenidos

El sistema es plenamente funcional y cumple con los objetivos propuestos: * Persistencia en Caliente: Los datos se guardan y recuperan de forma eficiente. * Sincronización Multiplataforma: Coherencia total de datos entre la web y el móvil. *



Automatización: Generación automática de rutinas y reportes de pagos.

5.2. Autoevaluación

El desarrollo presentó desafíos significativos, especialmente en la configuración de la seguridad de la API y el manejo de estados asíncronos en Android. La integración de JasperReports requirió una curva de aprendizaje adicional para el diseño de plantillas .jrxml. Sin embargo, estos obstáculos se superaron mediante la consulta de documentación oficial y foros especializados.

5.3. Conclusiones y Aprendizaje

Este proyecto ha consolidado mis conocimientos como desarrollador full-stack y multiplataforma. He aprendido la importancia de una API bien documentada y la necesidad de priorizar la experiencia de usuario (UX) tanto en entornos de escritorio como móviles. Profesionalmente, me siento capacitado para abordar arquitecturas cliente-servidor complejas.

6. BIBLIOGRAFÍA Y ANEXOS

6.1. Bibliografía

- Oracle. (2024). Java 17 Documentation. Recuperado de docs.oracle.com
- Spring Projects. (2024). Spring Boot Reference Guide. Recuperado de spring.io
- Google Developers. (2024). Android Developer Guides. Recuperado de developer.android.com



- MySQL. (2024). MySQL 8.0 Reference Manual. Recuperado de dev.mysql.com

6.2. Anexos: Manual de Usuario

El manual de usuario describe:

1. **Gestión de Socios:** Cómo dar de alta, baja o modificar datos de un cliente desde la web.
2. **Asignación de Rutinas:** Pasos para crear una rutina personalizada y asignarla a un socio.
3. **App Móvil:** Cómo el socio visualiza sus ejercicios diarios y el calendario de actividades.

6.3. Anexos: Manual Técnico

Fragmento de Código (Backend - Security Config)

@Configuration

@EnableWebSecurity

public class SecurityConfig {

 @Bean

 public SecurityFilterChain filterChain(HttpSecurity http) throws Exception

 { http.csrf(csrf -> csrf.disable())

 .authorizeHttpRequests(auth -> auth

 .requestMatchers("/api/auth/**").permitAll()

 .anyRequest().authenticated());

 return http.build();



```
}  
}
```

Diagrama de Base de Datos (Descripción) El sistema utiliza un esquema relacional donde la tabla usuarios gestiona el acceso, clientes almacena la información personal, vinculándose con pagos y rutinas mediante claves foráneas (cliente_id), garantizando que cada registro pertenezca a un socio único.